

Bom dia, professor. Vou apresentar minha dissertação de mestrado — *Explicando Decisões de LLMs via Otimização Inversa* — orientada pelo Prof. Raul Fonseca Neto. A apresentação está organizada em três blocos auto-contidos: no Bloco 1 o LLM é a fonte de decisões e nós recuperamos a métrica; no Bloco 2 o LLM é o aprendiz, recebendo rótulos de um perito externo; no Bloco 3 aplicamos tudo na base real de peso $\times$ altura.

LLMs são caixas-pretas. Quando se pede explicação em linguagem natural, recebe-se uma racionalização, não a causa real. Em aplicações críticas isso é insuficiente. A pergunta central: o LLM tem um processo decisório estável e aprendível? A proposta é inverter: em vez de pedir explicação, observar as decisões e recuperar matematicamente o critério por trás delas via otimização inversa.

Cinco hipóteses. H1 e H2 vivem no Bloco 1: o LLM mantém um critério estável e few-shot amplifica essa concordância. H3 e H4 vivem no Bloco 2: o LLM aprende critério de perito via in-context learning, e exemplos *hard* deveriam ensinar mais que *easy* — mas essa H4 foi refutada. H5 é transversal: resultados reproduzíveis entre sementes. Volto a cada uma ao longo da apresentação.

Trato cada classificação do LLM como saída de um classificador nearest-centroid ponderado por uma métrica de Mahalanobis diagonal. Dados pares ponto-rótulo do LLM, encontro $W \geq 0$ que reproduz suas decisões. Uso dois algoritmos independentes — Perceptron Estruturado e NNLS. Se ambos chegam ao mesmo W , a evidência não depende do método de estimação.

A distância é $d_W(x, c) = \sum w_j (x_j - c_j)^2$. Classificador: argmin sobre as classes. Conexão com Mahalanobis cheia: W diagonal corresponde a Σ^{-1} quando features são independentes. A escolha da forma diagonal é proposital: $O(d)$ parâmetros e interpretabilidade direta. Também destaco a nomenclatura nova das fases — Fase D é meia-lua (Bloco 1), Fase E é LLM aprendiz (Bloco 2).

Esta execução é um *smoke run* de validação com seed única 42, 1 repetição e n_shots [0, 5]. A produção usa 3 sementes vezes 3 repetições e n_shots até 40. O modelo é gpt-4o-mini com temperatura zero. Hiperparâmetros do Perceptron: $\eta = 0,001$ e $C = 1,0$, conforme o artigo CILAMCE 2017 que serve de referência.

O Perceptron Estruturado resolve um problema de margem com relaxação: maximizar γ sujeito a que a diferença de distâncias mais $\lambda\alpha_i$ seja maior que $\gamma\|w\|$, com $w \geq 0$. A regra de correção é a Equação 29 do artigo. Há busca binária em γ . O NNLS, por outro lado, é um problema convexo simples: $\min \|Aw - b\|^2$ com $w \geq 0$. Sem hiperparâmetros, determinístico, ótimo global.

Pipeline com 5 camadas de proteção: parser de 7 camadas, até 5 tentativas para respostas malformadas, fallback determinístico por hash MD5 (substituiu aritmética modular que introduzia viés geométrico), concorrência assíncrona com semáforo de 10 que reduz o tempo em uma ordem de magnitude, e timeout de 60s com wrap de 90s contra travamento.

Entramos no Bloco 1, onde o LLM é a fonte de decisões. O LLM rotula, nós aprendemos a métrica W a partir das decisões dele. Cobrirei os problemas A, B, C lineares e o D meia-lua. As hipóteses ligadas são H1 e H2.

Quatro problemas. Três lineares: A é o treino, B é rotação horária agressiva, C é anti-horária agressiva. Um não-linear: D é a meia-lua. As rotações de B e C agora são simetricamente agressivas com magnitude $\pm 1,5$ — decisão de 14/05/2026 para testar transferência em condições severas. A meia-lua entra como controle para testar não-linearidade implícita.

O Problema A é a bancada de treinamento. Gerado com `make_blobs`, dois centróides em $(-2, 0)$ e $(2, 0)$, separados 4 unidades na horizontal, com desvio padrão 1,2 isotrópico e 150 amostras balanceadas. É linearmente separável em x_1 , com fronteira ideal em $x_1 = 0$. Não é trivial pelo desvio ser próximo de metade da distância entre centróides, mas é fácil para o LLM.

B testa transferência da métrica em geometria deslocada agressivamente. Mesma função `make_blobs`, mas os centróides agora estão em $(-2; +1,5)$ e $(2; -1,5)$ — rotação horária. A magnitude $\pm 1,5$ é pareada simetricamente com C, e maior que a versão anterior $\pm 0,8$. Isso garante que B/C testam transferência em geometrias simetricamente desviadas, sem viesar para um lado.

C tem a mesma magnitude de B mas direção oposta: anti-horária. Centróides em $(-2; -1,5)$ e $(2; +1,5)$. O orientador pediu esta orientação oposta na reunião de 30/04 porque o C anterior parecia cópia de B com mais distorção. Agora B e C são pares: mesma severidade, direções opostas. Se houvesse viés sistemático do LLM ou da métrica, ele apareceria em apenas um dos dois.

A meia-lua entra no Bloco 1 como problema-controle não-linear. Usa `sklearn.datasets.make_moons` com $n = 150$, ruído 0,15 — valor clássico da literatura. O orientador disse, na reunião: *"a base da meia-lua você acha em qualquer lugar aí"*. É onde esperamos que augmentação R3/R4 ajude — e em A/B/C lineares não. É o problema-controle POSITIVO para a hipótese de não-linearidade.

Antes de aplicar Perceptron e NNLS ao LLM, faço um sanity check fundamental: e se o W verdadeiro fosse conhecido? Gero dados sintéticos rotulados por um W oráculo — por exemplo $[0,1; 2,0]$ — e checo se ambos os algoritmos recuperam esse W . Testo em três configurações anisotrópicas. Se falhasse aqui, qualquer achado sobre o LLM ficaria comprometido. É o piso de sanidade da metodologia.

A Fase A funciona assim: o LLM recebe 150 pontos do Problema A em zero-shot. Responde A ou B para cada. As 150 respostas formam (X, y_{LLM}) . Detalhe crucial: os centróides são recalculados *a partir das respostas do LLM*, não da classe verdadeira. Ancoramos a métrica no que o LLM ACHA serem as classes. Depois Perceptron e NNLS aprendem W independentemente, e medimos fidelidade.

Nesta execução com seed 42, a fidelidade ficou em 82% pelo Perceptron e 82,7% pelo NNLS — a métrica reproduz mais de quatro em cinco decisões do LLM. O gráfico mostra os pontos coloridos pelo erro — são basicamente os próximos da fronteira. Em produção com 3 seeds, o cosseno entre W_{perc} e W_{nnls} fica acima de 0,97.

A distribuição de W pelo Perceptron e NNLS. Em smoke run com seed única, o painel mostra apenas dois pontos no scatter. Em produção, com 3 seeds vezes 3 repetições, vemos 9 pontos por algoritmo e podemos verificar estabilidade. A razão w_1/w_2 é o que importa — a magnitude depende da escala da margem-alvo.

Aqui comparo Perceptron e NNLS lado a lado. O painel mostra fidelidade, consistências em B/C e scatter de W . Mensagem: os dois algoritmos são congruentes — chegam essencialmente ao mesmo W . Isso é importante porque significa que a evidência não depende do método de estimação escolhido. Se um detectasse algo que o outro não, teríamos um problema metodológico.

Aqui adianto o ponto que o orientador certamente vai questionar: fidelidade e acurácia são coisas diferentes. Fidelidade é o quanto a métrica reproduz o LLM. Acurácia é o quanto o LLM está correto contra o ground truth. Nesta execução a fidelidade é 82%. Em produção há casos em que o LLM inverte as classes e a acurácia cai para 28%, mas a fidelidade permanece 83%. Isso prova que medimos CONSISTÊNCIA INTERNA, não correção.

Detalhe metodológico que pode aparecer. Em B/C posso usar centróides LOCAIS (recalculados em B/C a partir das respostas do LLM em B) ou TRANSFERIDOS (reaproveitados do Problema A). Como padrão, uso LOCAIS — são mais fiéis à decisão real do LLM em cada problema. Os transferidos podem dar números melhores em B quando a geometria é próxima de A, mas isso seria "trapacear".

Com a nova rotação agressiva $\pm 1,5$: a Fase B ficou mais difícil (70% de consistência, Kappa 0,33), enquanto C surpreendentemente vai muito bem (89%, Kappa 0,77). O fenômeno faz sentido geométrico: a fronteira do LLM em A está mais alinhada com a rotação anti-horária. Em produção com 3 seeds, esta tendência deve se estabilizar; pode ser que C "ganhe" sempre, sugerindo um leve viés do LLM em uma direção.

Aqui aplico a augmentação de features — R2, R3, R4 — em A, B, C lineares. R3 adiciona x_1x_2 (hipérbole), R4 adiciona x_1^2 e x_2^2 (elipse). A expectativa é ganho zero, porque os problemas já são lineares. Esse é exatamente o achado: o ganho fica em torno de zero. Isso reforça a interpretação: R3/R4 só ajuda em não-linearidade real.

Agora na meia-lua, problema genuinamente não-linear. Aqui sim o ganho aparece. Em produção, R3 acrescenta cerca de 8,6 pontos percentuais de acurácia, e R4 outros 9,5. Contraste imediato com o slide anterior: R3/R4 ajuda APENAS onde há não-linearidade real. É o teste decisivo para a hipótese da não-linearidade implícita no critério do LLM.

Meta-teste cruzado. Aprende W em $R4$ na meia-lua, que recupera a fronteira em S . Aplica nos problemas lineares $A/B/C$ aumentados para $R4$. O esperado: queda forte na transferência. Isso reforça o achado do Oracle Validation: a métrica é específica do problema. Não existe um W universal independente dos centróides e da geometria.

A síntese visual mostra a tabela A/B/C lineares versus meia-lua: zero ganho nos lineares, ganho positivo no não-linear. É a forma mais condensada de comunicar a mensagem: ajuda quando precisa, não quando não precisa. Em produção o heatmap fica ainda mais claro com 3 seeds. Prepara o terreno para o Bloco 3, onde testaremos isso em base real.

Testei se a ordem das opções no prompt — "A ou B" versus "B ou A" — altera o resultado. A diferença é marginal, menos de 3 pontos percentuais em qualquer combinação. Existe, mas não é dominante. É importante registrá-lo para descartar como confound: nenhum dos achados principais depende dessa ordem.

Quatro variantes de prompt: default, *geometric* ("*você vê dois grupos de pontos*"), CoT ("*pense passo a passo*") e *tabular* (coordenadas em tabela markdown). Também testei três conjuntos de nomes para features: (x_1, x_2) , (altura, peso), (feature_1, feature_2). Em problemas sintéticos sem prior cultural, W é estável. Em problemas com prior, W pode mover — isso aparece com força na base real do Bloco 3.

Mostro o texto exato dos prompts default. Para a banca avaliar o experimento, é essencial ver o que de fato chega ao LLM, não só uma descrição em alto nível. O zero-shot tem três linhas; o few-shot adiciona cinco exemplos rotulados antes da pergunta. Trechos tomados diretamente do `llm_interactions.json`.

As três variantes lado a lado. O geometric prefixa com metáfora espacial. O CoT pede raciocínio passo a passo. O tabular formata exemplos como tabela markdown. Todos terminam com a mesma instrução final — "responda apenas A ou B" — para manter o sinal de saída controlado, variando só a forma de entrada.

Fechando o Bloco 1: H1 sustentada com Kappa 0,77–0,81 na Fase C; mais modesta na B com rotação agressiva. Fidelidade Fase A em torno de 82%. Perceptron e NNLS são congruentes. R3/R4 ajuda apenas no problema não-linear D meia-lua. Vieses marginais. É o quadro experimental do LLM como fonte de decisões.

Agora o Bloco 2: inversão completa de papéis. Um perito externo rotula, o LLM tenta reproduzir o critério dele a partir de exemplos few-shot. É o que chamamos de **Fase E**. Os problemas são E (linear com $W_{\text{expert}} = [0,3; 1,5]$) e F (meia-lua com perito = ground truth). Hipóteses: H3, H4 e H5.

Mostro lado a lado os dois problemas do Bloco 2: à esquerda o linear E com fronteira do expert quase horizontal (porque w_2 é cinco vezes maior que w_1); à direita a meia-lua com seu ground truth. Em ambos, o LLM precisa aprender a fronteira a partir de exemplos rotulados pelo perito.

O Problema E é o antigo "Problema D". É linear, mas o perito usa uma métrica anisotrópica $W = [0,3; 1,5]$ — o segundo eixo pesa cinco vezes mais. Isso produz uma fronteira quase horizontal, pouco intuitiva geometricamente. O LLM precisa CAPTURAR esse padrão a partir dos exemplos — não basta usar Euclidiana.

O Problema F usa a mesma meia-lua do Bloco 1, mas agora o perito é o próprio ground truth do `make_moons`. O LLM vê apenas (x_1, x_2) , sem augmentação R3 ou R4 no prompt. A pergunta é: consegue aprender a fronteira em S apenas com exemplos rotulados? Em produção, o ganho de 0 para 5-shot é particularmente forte aqui — cerca de 20 pontos percentuais.

A Fase E funciona assim: perito rotula os 150 pontos do problema; divido em treino e teste. Para cada n_{shot} — nesta execução apenas 0 e 5; em produção até 40 — seleciono exemplos via estratégia. Envio exemplos + ponto de teste em prompt few-shot. Meço acurácia, Kappa e F1 do LLM contra os rótulos do perito.

Quatro estratégias: *easy* (alta margem, longe da fronteira), *hard* (baixa margem, próximos da fronteira), *mixed* (50/50) e *random* (baseline). O gráfico mostra a distribuição espacial. A intuição de H4 era que *hard* ensina mais. Vamos ver no próximo slide que é o oposto.

As curvas de acurácia versus n_{shot} . Em smoke run com $n_{\text{shot}} \in \{0, 5\}$ a curva é curta. Em produção com até 40 exemplos, vemos o plateau clássico do in-context learning: ganho forte $0 \rightarrow 5$, depois desacelera. Isso sustenta H3.

Aqui aparece a refutação de H4. Mesmo no smoke com 5-shot, *easy* domina. Hard fica abaixo de easy, mixed e random. Em produção, com 3 seeds vezes 3 reps, o efeito é estatisticamente significativo com Cohen d cerca de 1,5. A intuição de active learning, importada de modelos com gradiente, não se aplica a LLMs in-context.

Em modelos com gradiente, exemplos ambíguos são informativos. Mas o LLM in-context não tem gradiente; ele usa exemplos como âncoras de classe. Pontos prototípicos são âncoras claras: *"é isto que A parece, é isto que B parece"*. Pontos ambíguos são confusos: *"este aqui é A? quase poderia ser B..."*. Isso confunde, não ajuda. Alinhado com Lyu et al. 2023.

Testei três experts diferentes. `aniso_x2` com $W = [0, 3, 1, 5]$, fronteira horizontal. `aniso_x1` com $W = [1, 5, 0, 3]$, vertical. `euclidean` com $W = [1, 1]$. Em produção, o LLM aprende todos, mas vai melhor no Euclidean — critério mais natural. Os dois anisotrópicos são mais difíceis.

Diluição testa: será que adicionar muitos easy "dilui" o sinal de hard fixos? Fixei 4 hard e adicionei easy progressivamente: 0, 2, 4, 10, 16, 20. A curva é monotônica crescente — adicionar easy ajuda sempre. Reforça o achado da H4: hard simplesmente não são informativos para LLM in-context.

Recency bias: a ordem dos exemplos few-shot importa? Testei quatro ordenações em diferentes n_{shot} . A variância entre ordenações é menor que 4 pontos percentuais. O viés existe mas é pequeno para o tamanho de prompt que usamos. Não explica resultados principais.

Agora a Fase E aplicada à meia-lua. O perito é o ground truth do `make_moons`. O LLM recebe apenas (x_1, x_2) , sem augmentação R3/R4 no prompt. A pergunta é: consegue capturar a fronteira em S a partir de exemplos? É o teste mais duro de H3, porque o problema é genuinamente não-linear.

Slide 45 — Ganho 0 para 5-shot na meia-lua

Em produção, o ganho 0→5-shot na meia-lua é da ordem de 20 pontos percentuais de acurácia. É o teste mais convincente de H3 — LLM realmente aprende a fronteira não-linear apenas com exemplos. CSV completo em `bloco23_external_phase_e`.

Comparo o LLM com k-NN, Logistic Regression e SVM, todos treinados nos MESMOS exemplos few-shot. O protocolo é fundamental: mesmos exemplos, mesmo conjunto de teste. Em produção, padrão: LLM compete em n_{shot} baixo, SVM e LR alcançam em n_{shot} alto. LLM não é universalmente superior.

Pedido específico do orientador na reunião em torno de 35 minutos: comparar LLM com Perceptron Estruturado treinado nos MESMOS exemplos do expert. Em produção, em peso $\times$ altura $n_{\text{shot}} = 5$, o Perceptron supera o LLM em 7 pontos percentuais (90% vs 83%). Quando há dados rotulados, métrica clássica é competitiva.

Fechando o Bloco 2: H3 sustentada com forte ganho 0→5-shot na meia-lua. H4 refutada com significância em n_{shot} pequeno; efeito some em 20. Baselines clássicos são competitivos com mais dados. Recency bias e viés de classes são marginais.

Bloco 3 é o estudo de caso real: peso $\times$ altura, base do orientador. É onde aplicamos tudo o que vimos até aqui em dados do mundo real, com 100 amostras e fronteira elíptica conhecida. A pergunta é: priors semânticos do LLM ajudam ou atrapalham?

A base foi enviada pelo orientador no e-mail das 19:15 do dia 30/04. São 100 amostras — 50 homens e 50 mulheres — com features já normalizadas no intervalo $[-1, 1]$ pelo formato do NeuroSolutions. O classificador ótimo bayesiano tem forma elíptica fornecida pelo professor.

A forma exata do classificador ótimo, dada pelo orientador, é $f(x_1, x_2) = x_2^2 - x_2 + x_1^2 - x_1 + \text{cte}$ — uma elipse. Por isso a augmentação R4 com (x_1, x_2, x_1^2, x_2^2) é perfeita aqui: a métrica linear em R4 recupera EXATAMENTE essa elipse. Limite teórico: classificador bayesiano de duas gaussianas com variâncias diferentes é quadrático.

Duas variantes de prompt. NEUTRA: features = (x_1, x_2) , LLM não sabe que é peso e altura. SEMÂNTICA: features = (peso, altura), LLM ativa priors. Pergunta em ambas: *homem ou mulher*. Replica em base real o experimento de nomes semânticos do Bloco 1.

Aprendo W em três configurações: $R2$, $R3$ com hipérbole, $R4$ com elipse alinhada. O gráfico tem quatro painéis: fidelidade pelo Perceptron, fidelidade pelo NNLS, acurácia contra o real pelo Perceptron, acurácia contra o real pelo NNLS. CSV completo disponível.

O e-mail 22:04 do orientador pediu explicitamente: *"apresentar também a acurácia da métrica para a rotulação da base de dados original"*. Atendo a esse pedido. Nesta execução a fidelidade da Fase A peso $\times$ altura ficou em 66%. Em produção, o gap fidelidade-acurácia cresce com o número de features — sinal do paradoxo.

O paradoxo é contra-intuitivo. Em peso $\times$ altura, ao passar de 2 para 4 features, a fidelidade sobe muito — a métrica decora melhor o LLM. Mas a acurácia contra o real cai. Como? Porque o LLM em si erra muito com prompt neutro, e a métrica com 4 pesos para 70 amostras de treino tem graus de liberdade suficientes para decorar o ruído das decisões erradas. Overfitting clássico.

Identificada a melhor configuração de features na Fase A, uso essa métrica como rotuladora para a Fase E. Os exemplos do prompt incluem o mesmo número de features. Reporto acurácia, Kappa e F1 do LLM ao longo de n_{shot} .

Curvas separadas por variante de prompt: (x_1, x_2) neutro versus (peso, altura) semântico, com dois painéis: LLM vs métrica e LLM vs real. Em produção, a variante semântica é sistematicamente melhor.

Slide 58 — LLM vs Perceptron na base real

Para cada n_{shot} , treino um Perceptron nos mesmos exemplos do expert. Em produção, em peso $\times$ altura $n_{\text{shot}} = 5$, o LLM com prompt semântico bate o Perceptron em alguns p.p. Mas com prompt neutro o Perceptron pode bater o LLM. Confirma que a vantagem do LLM vem dos priors, não do mecanismo de ICL.

Síntese: peso/altura semântico à frente em todos n_{shot} . x_1/x_2 neutro em n_{shot} médio, Perceptron ganha. Meia-lua em n_{shot} alto, LLM ganha. Interpretação: a vantagem do LLM vem dos priors do treinamento massivo. Mensagem honesta para a banca.

Atendendo ao adendo do e-mail das 22:06. Mostro lado a lado peso $\times$ altura com prompt semântico e neutro, ambos em 5-shot da seed 42. Os mesmos pontos são classificados de forma muito diferente. É a evidência visual mais direta do efeito do prior semântico.

Fechando o Bloco 3: priors semânticos são reais. R4 recupera a forma teórica mas overfit-a com 100 amostras. Paradoxo: fidelidade sobe, acurácia cai. LLM ganha contra Perceptron quando tem PRIORS e n_{shot} pequeno.

Tabela final cruzando os 3 blocos. A/B/C lineares: ganho zero (esperado). Meia-lua: ganho positivo (esperado). Peso $\times$ altura: ganho negativo (overfit, com 100 amostras não dá). Padrão: R3/R4 ajuda em não-linearidade real quando há dados suficientes.

Esta execução é smoke com 1 seed. Em produção, com 3 seeds, fidelidade fica em 83 ± 1 p.p., Consist B $\sim 78 \pm 2$, Consist C $\sim 90 \pm 3$. Os desvios são pequenos — H5 sustentada com folga. Nesta seed específica: Fid A = 82%, Consist B = 70%, Consist C = 89%.

Em smoke não há bootstrap por falta de amostras. Em produção: Few-shot vs zero-shot Cohen $d = 1,2$, Easy vs Hard $d = 1,5$. Cosseno entre $Ws \approx 0,98$. R2 vs R4 na meia-lua $d = 0,9$. Wilcoxon $p < 0,001$ em todos os principais.

Três testes adicionais. Point-biserial entre confiança do LLM e acerto: $r \approx 0,42$, $p < 0,001$. Mann-Whitney comparando margem em acertos vs erros: significativo. Kruskal-Wallis sobre ordem de exemplos: n.s., reforçando o achado de recency bias pequeno.

Síntese final por bloco. Bloco 1: H1 e H2 sustentadas. Bloco 2: H3 sustentada com ganho meia-lua, H4 refutada com significância. Transversal: H5 sustentada. Quatro das cinco sustentadas; H4 refutada com diagnóstico claro.

Cinco conclusões. Um: existe critério decisório estável no LLM, recuperável via otim. inversa. Dois: few-shot amplifica. Três: LLM aprende critério de perito via ICL, mas baselines clássicos são competitivos. Quatro: easy supera hard em few-shot pequeno. Cinco: kernel quadrático só ajuda em não-linearidade real E com dados suficientes.

Métrica diagonal não captura interações. 2D apenas. Peso $\times$ altura tem só 100 amostras. 1 LLM testado. Sem controle de versão do LLM entre execuções. Honestidade na apresentação das limitações é o que separa boa pesquisa de marketing.

Mahalanobis cheia, kernel não-diagonal. Bases UCI maiores. Múltiplos LLMs em paralelo — Claude e Gemini já estão integrados no código. Regularização L1 em W para seleção automática de features. Curva de aprendizado versus n_{feat} para escolher inflexão.

Encerro agradecendo o orientador Prof. Raul Fonseca Neto, o PPGCC da UFJF e minha família. Estou à disposição da banca para perguntas. Tenho dez slides de apêndice técnico cobrindo formulação matemática completa, overfitting, literatura e reprodutibilidade — inclusive o diagnóstico da busca binária de γ que o professor pediu.

Abro para perguntas da banca.

Para perguntas sobre o Perceptron Estruturado, este é o slide. Formulação primal Equação 28: $\max \gamma$ sujeito à restrição de margem com relaxação $\lambda \alpha_j$. Regra de correção Equação 29 atualiza w , α e α_j . Atualização da margem Equação 31. Hiperparâmetros do artigo: $\eta = 0,001$, $C \in [0,1; 1,0]$.

NNLS: $\min \|Aw - b\|^2$ com $w \geq 0$. Linha i de A tem diferenças de quadrados aos centróides. b é vetor de uns. Método de conjunto ativo de Lawson-Hanson de 1974. Vantagens sobre Perceptron: zero hiperparâmetros, determinístico, garante ótimo global do problema convexo.

Diferenças: Perceptron usa gradiente projetado, tem hiperparâmetros, não determinístico. NNLS usa QP via Lawson-Hanson. Mas na prática os dois são empiricamente equivalentes: cosseno entre W s acima de 0,97 em produção.

Caso emblemático: peso $\times$ altura com prompt neutro x_1/x_2 , 4 features. Fidelidade 78%, acurácia 37%. O LLM erra 63% dos casos, pior que chute. A métrica com 4 pesos e 70 amostras decora o ruído. Mitigações: L1/L2, mais dados, CV k -fold.

Trade-off clássico. 2 features tem alto bias mas baixa variância. 4 features tem baixo bias mas alta variância em $n = 100$. Rule of thumb 10 amostras por parâmetro sugere que 4 pesos está no limite com 70 amostras de treino.

Conexões com literatura. Ahuja & Orlin 2001: formulação clássica em PL. Schultz & Joachims 2003: margem SVM-style. Coelho, Borges, Fonseca Neto CILAMCE 2017: perceptron com relaxação — nosso algoritmo. Diferenciação: aplicação a LLMs, método black-box, independente de arquitetura.

Brown et al. 2020: GPT-3 e few-shot. Min et al. 2022: ground truth doesn't matter. Wei et al. 2022: emergent abilities. Lyu et al. 2023: prompts canônicos — alinhado com nossa refutação de H4.

Xing et al. 2002: original. Davis et al. 2007: ITML. Schultz & Joachims 2003: SVM-style margin. Justificativa da matriz diagonal: Liu et al. 2010, Kédem et al. 2012.

Comando único: `python src/dissertacao_mestrado.py`. Tudo controlado por constantes. Outputs incluem PNGs e CSVs com prefixos `bloco1`, `bloco2`, `bloco3` e `final`. Mostro aqui também o diagnóstico da busca binária em γ que o professor pediu na reunião em torno de 9 minutos: γ_{lo} monotônico crescente confirma convergência.

Honestamente: o LLM evolui silenciosamente entre execuções. Temperatura zero NÃO garante determinismo — batching em GPU produz diferenças residuais. Usamos 3 seeds vezes 3 reps em produção para 9 observações por configuração. Estocasticidade residual típica menor que 2% entre reps da mesma seed. É o que é factível hoje em pesquisa com LLMs comerciais.